

1. Dai File System ai Database Management Systems (DBMS)

I **File System** tradizionali presentano limitazioni strutturali fondamentali: **isolamento**, **duplicazione** e **rigidità** nella gestione e reperimento dei dati. Per superare questi limiti è stato introdotto il **DBMS**.

Cos'è un DBMS?

Un DBMS (*Database Management System*) **non è il database**, ma il **software** specializzato che consente la creazione, manipolazione, gestione e controllo dei dati all'interno di uno o più database.

Modello Logico vs. Modello Fisico

Un DBMS opera su due distinti livelli di astrazione:

- **Livello Logico:** Rappresenta la struttura dei dati così come viene vista ed elaborata dall'utente/programmatore (es. tabelle, campi, chiavi).
- **Livello Fisico:** Corrisponde alla reale memorizzazione binaria e all'organizzazione dei dati sui dispositivi di archiviazione secondaria (disco rigido, SSD).

Vantaggi Fondamentali dell'uso di un DBMS

1. **Riduzione della Ridondanza e Normalizzazione:** Minimizza la duplicazione inutile dei dati tramite tecniche di normalizzazione (tecnica analizzata dopo la modellazione logica per identificare e rimuovere anomalie e inconsistenze).
2. **Integrità dei Dati:** Garantisce la qualità e l'accuratezza dei dati tramite l'applicazione di vincoli di integrità:
 - a. **Vincoli di Dominio:** Limitano i valori che un singolo campo può assumere (es. tipo di dato, intervallo numerico).
 - b. **Vincoli di Integrità Referenziale:** Regolano le relazioni tra tabelle diverse tramite l'uso di chiavi esterne (*Foreign Key*). Impediscono l'esistenza di dati "orfani" (es. un ordine associato a un ID cliente inesistente).
 - c. **Azioni Referenziali (es. ON DELETE CASCADE):** Regole che definiscono il comportamento del sistema quando un dato padre viene eliminato o aggiornato.
3. **Controllo della Concorrenza e Transazioni:** Gestisce l'accesso simultaneo di più utenti agli stessi dati senza causare conflitti o corruzioni.

- a. **Transazione:** Blocco di istruzioni SQL eseguito in maniera **atomica** (tutto o niente).
 - b. Le transazioni devono rispettare le proprietà **ACID**: *Atomicità, Consistenza, Isolamento (Isolation), Durabilità*.
4. **Sicurezza e Controllo degli Accessi:** Gestione granulare dei permessi tramite **ruoli e privilegi** (es. consentire a un gruppo di utenti la sola operazione di lettura e ad altri anche la scrittura).
5. **Efficienza Temporale e Spaziale:**
 - a. **Temporale:** Utilizzo di indici e ottimizzatori di query per restituire risultati in pochi millisecondi rispetto alle ricerche manuali su file o archivi cartacei.
 - b. **Spaziale:** Ottimizzazione dell'occupazione del disco fisso.
6. **Backup e Recovery:** Strumenti integrati per il ripristino dei dati in caso di guasti hardware o disastri.

Esempio Pratico: Gestione della Concorrenza vs. Overbooking

- **Gestione Corretta con Transazioni (es. Biglietteria Eventi):**

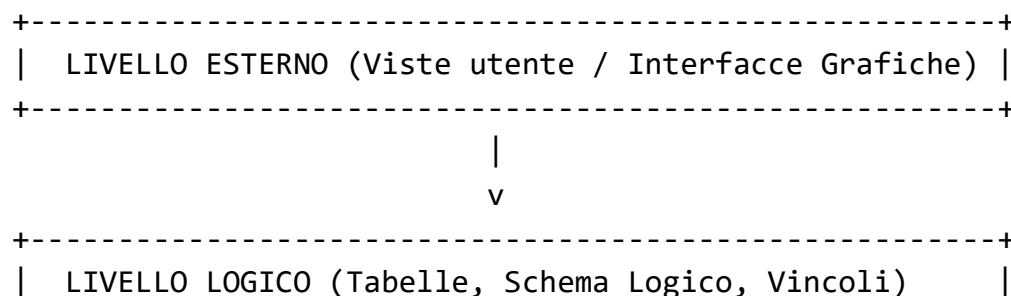
Quando un utente seleziona un posto su TicketOne, il DBMS avvia una transazione e imposta un *countdown*. Il posto viene temporaneamente "bloccato" (isolato da altri utenti). Se l'acquisto va a buon fine, la transazione fa il *commit*; in caso contrario, fa il *rollback* e libera la risorsa.

- **Gestione Senza Transazioni Strette (es. Overbooking Aereo):**

Le compagnie aeree sfruttano di proposito la concorrenza aperta senza blocchi rigidi per vendere più biglietti dei posti reali sul volo, basandosi sulla probabilità statistica che alcuni passeggeri non si presentino al gate (*no-show*).

2. Architettura ANSI/SPARC

Standard teorico proposto nel 1975 che definisce un'architettura a **tre livelli di astrazione** per disaccoppiare le applicazioni utente dalla struttura fisica dei dati.



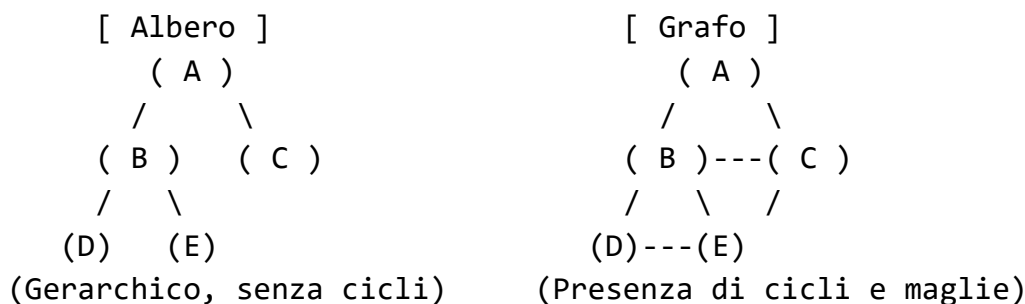
- **Documentali (es. MongoDB):** I dati sono strutturati come documenti flessibili (tipicamente in formato **JSON** o BSON). Non richiedono uno schema rigido preventivo: documenti appartenenti allo stesso insieme possono avere campi diversi.
- **Column-Oriented / Column-Family (es. Apache Cassandra):** Progettati per la gestione scalabile di enormi moli di dati (*Big Data*) memorizzando le informazioni per colonne anziché per righe.

- **A Grafo (es. Neo4j):** Basati sulla teoria dei grafi per modellare dati ad alta interconnessione.

Esempio Pratico: Strutture Dati a Grafo vs. Albero

In informatica, alberi e grafi sono strutture dati non lineari:

- **Albero:** Struttura gerarchica in cui ogni nodo ha un solo nodo padre (tranne la radice). Non sono ammessi cicli.
- **Grafo:** Insieme di nodi (vertici) e archi (collegamenti). Può contenere **cicli** e un nodo può avere più connessioni/padri.



I DB a Grafo (come Neo4j) sono ideali per social network (es. "amico di un amico") o sistemi di raccomandazione.

5. Modellazione Concettuale: Il Modello Entity-Relationship (ER)

Definizione Matematica di Relazione

Dal punto di vista matematico, una **Relazione** R tra due insiemi di entità A e B è un **sottoinsieme del prodotto cartesiano** $A \times B$.

Esempio Pratico: Prodotto Cartesiano e Relazione

Dati due insiemi:

- $A = \{A_1, A_2\}$ (Insieme degli Studenti)
- $B = \{B_1, B_2, B_3\}$ (Insieme dei Corsi)

Il prodotto cartesiano $A \times B$ rappresenta tutte le combinazioni possibili:

$$A \times B = \{(A_1, B_1), (A_1, B_2), (A_1, B_3), (A_2, B_1), (A_2, B_2), (A_2, B_3)\}$$

La relazione reale nel database è un sottoinsieme di questo prodotto cartesiano, poiché un singolo studente non è iscritto contemporaneamente a tutti i corsi esistenti, ma solo ad alcuni di essi:

$$R = \{(A_1, B_1), (A_2, B_3)\} \subseteq A \times B$$

Nelle occorrenze di una relazione non possono esistere tuple (coppie o n-ple) duplicate.

Componenti del Diagramma ER

1. Entità e Relazioni

- **Entità:** Rappresentano classi di oggetti omogenei del mondo reale (es. Studente, Corso).
- **Relazione/Associazione:** Legame logico tra due o più entità.

Ristrutturazione Modello: Se una relazione semplice non è sufficiente a modellare un problema (es. uno studente che tenta lo stesso esame in più appelli differenti), la relazione viene reificata, ossia trasformata in un'**entità intermedia** (es. l'entità Sostenerimento o Appello).

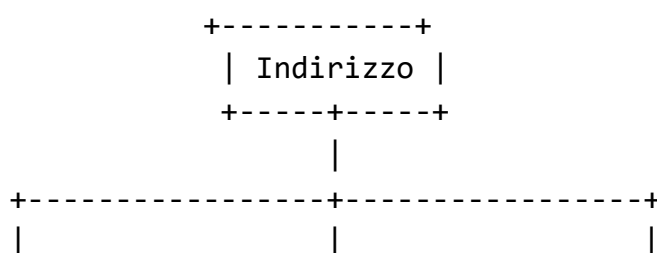
2. Attributi

Caratteristiche elementari delle entità o delle relazioni.

- **Semplici:** Valore atomico (es. Nome, Cognome, Eta). Si disegnano con un pallino vuoto.
- **Composti:** Attributi suddivisi in sotto-proprietà più specifiche per facilitarne la gestione logica.

Esempio di Attributo Composto

Invece di definire un attributo unico Indirizzo (stringa generica), si definisce un attributo composto costituito da un insieme di attributi semplici:



$\vee$
(Via)

$\vee$
(Civico)

$\vee$
(CAP)

- **Multi-valore:** Attributi che possono contenere più di un valore per una singola occorrenza (es. Telefono con cardinalità $(0,N)$).

3. *Identificatori (Chiavi Candidate)*

Rappresentano l'insieme di attributi che identificano in modo univoco un'occorrenza di un'entità. Si rappresentano graficamente con un **pallino pieno**.

- **Identificatori Interni:**
 - **Semplici:** Un singolo attributo (es. Matricola o Email).
 - **Composti:** Un'unione di due o più attributi collegati tra loro (es. Nome + Cognome).
- **Identificatori Esterni:** Utilizzano attributi di altre entità collegate tramite relazioni (analizzati nelle fasi successive di progettazione).

Nota: Le **relazioni non hanno identificatori propri**; la loro univocità è data dagli identificatori delle entità che vi partecipano.